Overview of Masked language model

Subject: Masked language model

1.

Cost BERT was trained on the BookCorpus (800M words) and a filtered version of English Wikipedia (2,500M words) without lists, tables, and headers. Training BERTBASE on 4 cloud TPU (16 TPU chips total) took 4 days, at an estimated cost of 500 USD. Training BERTLARGE on 16 cloud TPU (64 TPU chips total) took 4 days.

2.

replaced with a [MASK] token with probability 80%, replaced with a random word token with probability 10%, not replaced with probability 10%. The reason not all selected tokens are masked is to avoid the dataset shift problem. The dataset shift problem arises when the distribution of inputs seen during training differs significantly from the distribution encountered during inference. A trained BERT model might be applied to word representation (like Word2Vec), where it would be run over sentences not containing any [MASK] tokens. It is later found that more diverse training objectives are generally better. As an illustrative example, consider the sentence "my dog is cute". It would first be divided into tokens like "my1 dog2 is3 cute4". Then a random token in the sentence would be picked. Let it be the 4th one "cute4". Next, there would be three possibilities:

3.

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.

4.

Architectural family The encoder stack of BERT has 2 free parameters: L {\displaystyle L} , the number of layers, and H {\displaystyle H} , the hidden size. There are always H / 64 {\displaystyle H/64} self-attention heads, and the feed-forward/filter size is always 4 H {\displaystyle 4H} . By varying these two numbers, one obtains an entire family of BERT models. For BERT:

5.

History BERT was originally published by Google researchers Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. The design has its origins from pre-training contextual

representations, including semi-supervised sequence learning, generative pre-training, ELMo, and ULMFit. Unlike previous models, BERT is a deeply bidirectional, unsupervised language representation, pre-trained using only a plain text corpus. Context-free models such as word2vec or GloVe generate a single word embedding representation for each word in the vocabulary, whereas BERT takes into account the context for each occurrence of a given word. For instance, whereas the vector for "running" will have the same word2vec vector representation for both of its occurrences in the sentences "He is running a company" and "He is running a marathon", BERT will provide a contextualized embedding that will be different according to the sentence. On October 25, 2019, Google announced that they had started applying BERT models to English-language search queries on Google Search within the US. On December 9, 2019, it was reported that BERT had been adopted by Google Search for over 70 languages. In October 2020, almost every single English-based query was processed by a BERT model.

6.

Embedding This section describes the embedding used by BERTBASE. The other one, BERTLARGE, is similar, just larger. The tokenizer of BERT is WordPiece, which is a sub-word strategy like byte-pair encoding. Its vocabulary size is 30,000, and any token not appearing in its vocabulary is replaced by [UNK] ("unknown").

7.

Interpretation Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. Their performance on these natural language understanding tasks are not yet well understood. Several research publications in 2018 and 2019 focused on investigating the relationship behind BERT's output as a result of carefully chosen input sequences, analysis of internal vector representations through probing classifiers, and the relationships represented by attention weights. The high performance of the BERT model could also be attributed to the fact that it is bidirectionally trained. This means that BERT, based on the Transformer model architecture, applies its self-attention mechanism to learn information from a text from the left and right side during training, and consequently gains a deep understanding of the context. For example, the word fine can have two different meanings depending on the context (I feel fine today, She has fine blond hair). BERT considers the words surrounding the target word fine from the left and right side. However it comes at a cost: due to encoder-only architecture lacking a decoder, BERT can't be prompted and can't generate text, while bidirectional models in general do not work effectively without the right side, thus being difficult to prompt. As an illustrative example, if one wishes to use BERT to continue a sentence fragment "Today, I went to", then naively one would mask out all the tokens as "Today, I went to [MASK] [MASK] [MASK] ... [MASK] ." where the number of [MASK] is the length of the sentence one wishes to extend to. However, this constitutes a dataset shift, as during training, BERT has never seen sentences with that many tokens masked out. Consequently, its performance degrades. More sophisticated techniques allow text generation, but at a high computational cost.

8.

with probability 80%, the chosen token is masked, resulting in "my1 dog2 is3 [MASK]4"; with probability 10%, the chosen token is replaced by a uniformly sampled random token, such as "happy", resulting in "my1 dog2 is3 happy4"; with probability 10%, nothing is done, resulting in "my1 dog2 is3

cute4". After processing the input text, the model's 4th output vector is passed to its decoder layer, which outputs a probability distribution over its 30,000-dimensional vocabulary space.